Ranking Function

Baza danych - POSTGRESQL

Przemysław Starosta - 2026

Row_number

Funkcja **row_number()** jest funkcją okienkową w SQL, która przypisuje unikalny, sekwencyjny numer każdemu wierszowi w zestawie wyników, zgodnie z określonym porządkiem sortowania. Jej główne zastosowania to:

- **Paginacja danych:** Umożliwia podział dużego zbioru wyników na mniejsze strony.

- **Ranking:** Służy do tworzenia rankingów lub porządkowania wierszy według określonych kryteriów.

- **Eliminacja duplikatów:** Może pomóc w identyfikacji i usunięciu zduplikowanych rekordów, poprzez wybranie tylko jednego wiersza dla danej grupy.

Row_number

	last_name	salary	row_number
1	Olson	2,100	1
2	Markle	2,200	2
3	Philtanker	2,200	3
4	Gee	2,400	4
5	Landry	2,400	5
6	Patel	2,500	6
7	Colmenares	2,500	7
8	Sullivan	2,500	8
9	Vargas	2,500	9

```
SELECT
    last_name,
    salary,
    row_number() over (order by salary)
    row_number
FROM
        employees
```

Partition

Operator **PARTITION BY** w funkcjach okienkowych w SQL służy do podziału zbioru wyników na oddzielne grupy, zwane partycjami. Dzięki temu funkcje okienkowe (takie jak ROW_NUMBER, RANK, SUM, AVG, itp.) wykonują obliczenia niezależnie dla każdej z tych partycji.

Kluczowe aspekty:

- **Izolacja obliczeń:** funkcja okienkowa oblicza swoje wyniki w obrębie każdej partycji osobno. Na przykład, używając PARTITION BY department, możesz obliczyć ranking pracowników w ramach każdego działu z osobna.
- **Bez zmiany liczby wierszy:** w przeciwieństwie do klauzuli GROUP BY, PARTITION BY nie redukuje liczby zwracanych wierszy. Każdy wiersz pozostaje w zbiorze wyników, ale funkcja okienkowa operuje tylko na wierszach należących do tej samej partycji.

Row_numer - partition

	last_name	job_id	salary	row_number
1	Gietz	AC_ACCOUNT	8,300	1
2	Higgins	AC_MGR	12,000	1
3	Whalen	AD_ASST	4,400	1
4	King	AD_PRES	24,000	1
5	De Haan	AD_VP	17,000	1
6	Kochhar	AD_VP	17,000	2
7	Popp	FI_ACCOUNT	6,900	1
8	Sciarra	FI_ACCOUNT	7,700	2
9	Urman	FI_ACCOUNT	7,800	3
10	Chen	FI_ACCOUNT	8,200	4

```
SELECT
    last_name,
    job_id,
    salary,
    row_number()
        over (partition by job_id order by salary ) row_number
FROM
        employees
```

Rank

Funkcje **RANK()** oraz **DENSE_RANK()** to funkcje okienkowe w SQL, które służą do przydzielania pozycji (rang)
w zestawie wyników według określonego kryterium sortowania. Oto, jak działają i jakie mają zastosowania:

RANK()

• **Opis:**
Funkcja **RANK()** przypisuje rangę wierszom na podstawie podanego porządku.
W przypadku, gdy dwa lub więcej wierszy ma tę samą wartość sortującą, otrzymują one tę samą rangę.

• **Przerwy w rankingu:**
Po wystąpieniu remisów, kolejne rangi są pomijane. Na przykład, jeśli dwa rekordy uzyskają rangę 1, to kolejny rekord otrzyma rangę 3, a nie 2.

• **Przykładowe zastosowanie:**
Ranking pracowników według wynagrodzenia, gdzie przy równych wynagrodzeniach pracownicy mają tę samą pozycję, ale po nich następuje luka w kolejności.

Rank

	last_name	salary	rank
1	King	24,000	1
2	Kochhar	17,000	2
3	De Haan	17,000	2
4	Russell	14,000	4
5	Partners	13,500	5
6	Hartstein	13,000	6
7	Higgins	12,000	7
8	Errazuriz	12,000	7
9	Greenberg	12,000	7
10	Ozer	11,500	10

```
SELECT
    last_name,
    salary,
    rank() over (order by salary desc) rank
FROM
        employees
```

Rank
- partition

	last_name	job_id	salary	partition_rank
1	Gietz	AC_ACCOUNT	8,300	1
2	Higgins	AC_MGR	12,000	1
3	Whalen	AD_ASST	4,400	1
4	King	AD_PRES	24,000	1
5	De Haan	AD_VP	17,000	1
6	Kochhar	AD_VP	17,000	1
7	Popp	FI_ACCOUNT	6,900	1
8	Sciarra	FI_ACCOUNT	7,700	2
9	Urman	FI_ACCOUNT	7,800	3
10	Chen	FI_ACCOUNT	8,200	4
11	Faviet	FI_ACCOUNT	9,000	5

```
SELECT
    last_name,
    job_id,
    salary,
    rank()
    over (partition by job_id order by salary) partition_rank
FROM
        employees
```

Dense_rank

DENSE_RANK()

- **Opis:**
Funkcja **DENSE_RANK()** również przypisuje rangę wierszom, ale w odróżnieniu od **RANK()**, nie pomija kolejnych wartości.

- **Brak przerw:**
Przy remisach wszystkie rekordy otrzymują tę samą rangę, a kolejny unikalny rekord dostaje następną liczbę w sekwencji. Na przykład, jeśli dwa rekordy mają rangę 1, to następny rekord otrzyma rangę 2.

- **Przykładowe zastosowanie:**
Gdy potrzebujemy ciągłego rankingu bez luk, co jest przydatne w raportach lub przydzielaniu pozycji w rankingach.

Dense_rank

	last_name	salary	dense_rank
1	Olson	2,100	1
2	Markle	2,200	2
3	Philtanker	2,200	2
4	Gee	2,400	3
5	Landry	2,400	3
6	Patel	2,500	4
7	Colmenares	2,500	4
8	Sullivan	2,500	4
9	Vargas	2,500	4

```
SELECT
    last_name,
    salary,
    dense_rank() over (order by salary) dense_rank
FROM
        employees
```

Dense_rank - partition

	last_name	job_id	salary	partition_dense_rank
1	Gietz	AC_ACCOUNT	8,300	1
2	Higgins	AC_MGR	12,000	1
3	Whalen	AD_ASST	4,400	1
4	King	AD_PRES	24,000	1
5	De Haan	AD_VP	17,000	1
6	Kochhar	AD_VP	17,000	1
7	Popp	FI_ACCOUNT	6,900	1
8	Sciarra	FI_ACCOUNT	7,700	2
9	Urman	FI_ACCOUNT	7,800	3
10	Chen	FI_ACCOUNT	8,200	4
11	Faviet	FI_ACCOUNT	9,000	5

```
SELECT
    last_name,
    job_id,
    salary,
    dense_rank()
        over (partition by job_id order by salary) partition_dense_rank
FROM
        employees
```

Rank vs Dense_rank

	last_name	salary	rank	dense_rank
1	Olson	2,100	1	1
2	Markle	2,200	2	2
3	Philtanker	2,200	2	2
4	Gee	2,400	4	3
5	Landry	2,400	4	3
6	Patel	2,500	6	4
7	Colmenares	2,500	6	4
8	Sullivan	2,500	6	4
9	Vargas	2,500	6	4

```
SELECT
    last_name,
    salary,
    rank() over (order by salary) rank,
    dense_rank() over (order by salary)
    dense_rank
FROM
        employees
```

Comparision - partition

	last_name	job_id	salary	partition_rank	partition_dense_rank
13	Mavris	HR_REP	6,500	1	1
14	Hunold	IT_PROG	9,000	1	1
15	Ernst	IT_PROG	6,000	2	2
16	Austin	IT_PROG	4,800	3	3
17	Pataballa	IT_PROG	4,800	3	3
18	Lorentz	IT_PROG	4,200	5	4
19	Hartstein	MK_MAN	13,000	1	1
20	Fay	MK_REP	6,000	1	1
21	Baer	PR_REP	10,000	1	1

```
SELECT
    last_name,
    job_id,
    salary,
    rank() over (partition by job_id order by salary desc) partition_rank,
    dense_rank() over (partition by job_id order by salary desc)
    partition_dense_rank
FROM
        employees
```

Cume_dist (with current row)

Funkcja CUME_DIST() (cumulative distribution) jest funkcją okienkową w SQL, która zwraca skumulowany rozkład danego wiersza w obrębie określonego okna (partycji) w zbiorze danych. Mówiąc prościej, oblicza ona, jaki ułamek (procent) wszystkich wierszy w danej grupie (oknie) ma wartość mniejszą lub równą wartości bieżącego wiersza.

Jak działa CUME_DIST()

- **Porządkowanie wierszy** - w pierwszym kroku określamy, według jakich kolumn chcemy porządkować wiersze, np. ORDER BY Salary DESC.
- **Wyznaczenie rozkładu skumulowanego** - dla każdego wiersza obliczany jest odsetek wierszy, które mają wartość mniejszą lub równą danej wartości.
    - Jeśli wiersz ma najwyższą wartość, np. największe wynagrodzenie, to CUME_DIST() zwykle zwróci niewielką wartość (zależy, ile osób ma taką samą wartość).
    - Jeśli wiersz ma najniższą wartość, wynik będzie najbliższy 1 (czyli 100%).
- **Wartości zwracane** - funkcja zwraca wartości z zakresu (0, 1], co interpretujemy jako procent skumulowany (przedział od powyżej 0 do 1 włącznie).

Cume_dist (with current row)

	123 val	123 cume_dist
1	100	0.25
2	200	0.5
3	300	0.75
4	600	1

```
select
t1. val,
cume_dist() over (order by t1.val)
cume_dist
from(
    SELECT
    100 as val
    union
    SELECT
    200 as val
    union
    SELECT
    300 as val
    union
    SELECT
    600 as val ) t1
```

Percent_rank (without current row)

Funkcja **PERCENT_RANK()** jest funkcją okienkową w SQL, która zwraca względną pozycję (rangę) danego wiersza w danym zbiorze (lub partycji) w postaci wartości od **0** do **1**. Innymi słowy, określa, gdzie wiersz plasuje się procentowo wśród wszystkich wierszy w obrębie ustalonego porządku sortowania.

Jak działa PERCENT_RANK()

- **Określenie kolejności (ORDER BY)** -najpierw definiuje się porządek sortowania (np. według kolumny Salary).
- **Obliczanie rankingu** - PERCENT_RANK() korzysta z funkcji RANK() do wyznaczenia rangi wiersza, a następnie przekształca tę rangę w przedział 0,1 według wzoru:
  - Pierwszy wiersz (o rank = 1) otrzyma zawsze wartość 0,
  - Ostatni wiersz (o rank równej liczbie wierszy w partycji) otrzyma wartość 1 (o ile w partycji jest więcej niż jeden wiersz).
- **Zakres zwracanych wartości**
  - 0 – oznacza, że dany wiersz jest na samym początku w danej kolejności (najniższa wartość sortująca),
  - 1 – oznacza, że wiersz jest na końcu (najwyższa wartość sortująca) i jest zarazem ostatni w danej partycji.
  - Dla wierszy pomiędzy początkiem i końcem – wartości są rozłożone w przedziale (0, 1), proporcjonalnie do ich rangi.

Percent_rank (without current row)

	val	percent_rank
1	100	0
2	200	0.3333333333
3	300	0.6666666667
4	600	1

```
select
t1. val,
percent_rank() over (order by t1.val)
percent_rank
from(
        SELECT
        100 as val
        union
        SELECT
        200 as val
        union
        SELECT
        300 as val
        union
        SELECT
        600 as val ) t1
```

Percent_rank vs Cume_dist

Różnica między PERCENT_RANK() a CUME_DIST()

• **CUME_DIST()** wyznacza, jaki ułamek (procent) wierszy ma wartość **mniejszą lub równą** bieżącemu wierszowi.

• **PERCENT_RANK()** przelicza **RANK()** na skalę od 0 do 1, co oznacza pozycję bieżącego wiersza w uporządkowanym zbiorze w ujęciu procentowym (z uwzględnieniem możliwych “skoków” przy remisach).

Wybór między tymi funkcjami zależy od potrzeb analitycznych:

• **PERCENT_RANK()** dobrze pokazuje, w jakim “procentowym” miejscu znajduje się dany wiersz w zbiorze (pierwszy wiersz zawsze 0, ostatni zawsze 1).

• **CUME_DIST()** wskazuje, jaki odsetek wierszy osiąga **co najwyżej** daną wartość (pierwszy wiersz może otrzymać wartość większą niż 0, jeśli wiele wierszy ma tę samą wartość).

Ntile

Funkcja **NTILE()** w SQL jest funkcją okienkową, która dzieli zbiór wyników na określoną liczbę przedziałów (tzw. „kubełków” lub „wiaderek”). Dzięki temu możemy podzielić wiersze w uporządkowanym zestawie danych na równe (lub prawie równe) grupy.

Jak działa NTILE?

- **Określenie liczby przedziałów (n**) - w wywołaniu funkcji podajemy liczbę n, czyli liczbę przedziałów, na jakie chcemy podzielić wyniki.

  NTILE(n) OVER (ORDER BY ...)

- **Porządkowanie wierszy** - najpierw określamy, w jakiej kolejności sortujemy dane w oknie (np. ORDER BY Salary DESC).
- **Przydzielanie do przedziałów** -następnie wiersze są rozdzielane na n grup. Każdy wiersz dostaje numer grupy w przedziale 1 do n.
  - Jeśli zbiór dzieli się równo, to każda grupa ma tyle samo wierszy.
  - Jeśli nie dzieli się równo, to niektóre grupy mogą mieć o jeden wiersz więcej.
- **Wynik** - zwracana kolumna (z funkcji NTILE) zawiera numer grupy.

Ntile

	last_name	job_id	salary	ntile
1	Lorentz	IT_PROG	4,200	1
2	Austin	IT_PROG	4,800	1
3	Pataballa	IT_PROG	4,800	1
4	Ernst	IT_PROG	6,000	2
5	Popp	FI_ACCOUNT	6,900	2
6	Sciarra	FI_ACCOUNT	7,700	2
7	Urman	FI_ACCOUNT	7,800	3
8	Chen	FI_ACCOUNT	8,200	3
9	Faviet	FI_ACCOUNT	9,000	4
10	Hunold	IT_PROG	9,000	4

```
SELECT
    last_name,
    job_id,
    salary,
    ntile(4) over (order by salary ) ntile
FROM
        employees
where
        job_id in ('FI_ACCOUNT', 'IT_PROG')
```

Ntile - partiion

	last_name	job_id	salary	ntile
1	Popp	FI_ACCOUNT	6,900	1
2	Sciarra	FI_ACCOUNT	7,700	1
3	Urman	FI_ACCOUNT	7,800	2
4	Chen	FI_ACCOUNT	8,200	3
5	Faviet	FI_ACCOUNT	9,000	4
6	Lorentz	IT_PROG	4,200	1
7	Pataballa	IT_PROG	4,800	1
8	Austin	IT_PROG	4,800	2
9	Ernst	IT_PROG	6,000	3
10	Hunold	IT_PROG	9,000	4

```
SELECT
    last_name,
    job_id,
    salary,
    ntile(4) over (partition by job_id order by salary )
    ntile
FROM
        employees
where
        job_id in ('FI_ACCOUNT', 'IT_PROG')
```

Widht_bucket

Funkcja **width_bucket** służy do rozdzielania zakresu wartości liczbowych na określoną liczbę równych "kubełków" (ang. buckets) lub przedziałów. Jest to bardzo przydatne w analizie danych, gdy chcesz pogrupować dane w histogramy lub kategoryzować wartości w dyskretne przedziały..

width_bucket(operand, min_value, max_value, num_buckets)

Parametry

- **operand**: Wartość liczbowa, którą chcesz skategoryzować. Może to być kolumna, stała lub wyrażenie.
- **min_value**: Dolna granica zakresu, który ma być podzielony na kubełki.
- **max_value**: Górna granica zakresu, który ma być podzielony na kubełki.
- **num_buckets**: Liczba kubełków (przedziałów), na które ma być podzielony zakres od min_value do max_value.

Widht_bucket

Jak działa width_bucket?

- Funkcja width_bucket zwraca numer kubełka, do którego należy operand. Kubełki są numerowane od 1 do num_buckets.
  - **Kubełek 0:** Jeśli operand jest mniejszy niż min_value, funkcja zwraca 0.
  - **Kubełki 1 do num_buckets:** Jeśli operand mieści się w zakresie [min_value, max_value], funkcja zwraca numer kubełka, do którego należy.
  - **Kubełek num_buckets + 1**: Jeśli operand jest większy lub równy max_value, funkcja zwraca num_buckets + 1.

WIDTH_BUCKET

	last_name	job_id	salary	width_bucket
1	King	AD_PRES	24,000	5
2	Kochhar	AD_VP	17,000	4
3	De Haan	AD_VP	17,000	4
4	Hunold	IT_PROG	9,000	2
5	Ernst	IT_PROG	6,000	1
6	Austin	IT_PROG	4,800	0
7	Pataballa	IT_PROG	4,800	0
8	Lorentz	IT_PROG	4,200	0
9	Greenberg	FI_MGR	12,000	2

```
SELECT
    last_name,
    job_id,
    salary,
    width_bucket (salary, 5000, 20000, 4)
FROM
        employees
```